



Observability & AI Ops Stack

AICB-P2T2 · Ngày 23 · Lecture ·
MLOps · LLMOps · AIOps · AgentOps

Giảng viên

VinUniversity · Phase 2 · Track 2 · Tuần 5

HÃY SUY NGHĨ...

*“Model chạy tốt hôm qua — hôm nay accuracy drop 20%. Bạn phát hiện được không, và bao lâu? **Reality check:** Phần lớn team AI chỉ biết model fail khi user phàn nàn — trung bình mất 3–5 ngày. Observability không phải nice-to-have — đó là survival.”*

Giữ câu hỏi này trong đầu khi học bài hôm nay

- §1–2 Evolution · Pillars (RED/USE)
- §3–4 Prometheus · TSDB/Storage
- §5–6 Grafana/Loki · SLO/Burn-rate
- §7 Distributed Tracing & OTel
- §8–9 **GPU-Serving** · **eBPF/Profiling**
- §10–11 Drift/Eval · Cost/FinOps
- §12 Postmortems · §13–14 **Agent-obs / Harness** · §15 Experimentation

- §16 **Ops Trinity** (DevOps→ML/LLM/Agent)
- §17 **MLOps** Deepdive
- §18 **LLMOps** Deepdive
- §19 **AgentOps** Deepdive
- §20 **AI Ops** — AI cho IT Ops
- §21 Demo + Lab + Tổng Kết

Cả ngày là một mạch: *quan sát hệ* → *quan sát agent tự hành* → *vận hành cả vòng đời*. Observability là table-stakes; eval + governance là mặt trận tiếp theo.

Day-12/13 (Nền tảng) đã dạy *cái gì* là metric/log/trace/SLO. Track Infra này đi **sâu & thực chiến**:

1. Vận hành observability ở quy mô: Prom 3.x/TSDB/columnar, GPU+vLLM telemetry (goodput), eBPF + profiling, drift/eval, FinOps, agent-obs, harness/flywheel — §1–14
2. **MLOps**: CI/CD/CT, registry+lineage, maturity, platforms — §17
3. **LLMOps**: prompt-as-code, eval-driven dev, RAGOps, fine-tune ops, gateway — §18
4. **AgentOps**: agent reliability (pass^k), memory/MCP/A2A ops, frameworks, governance — §19
5. Nổi ba mạch: *quan sát hệ* → *quan sát agent* → *vận hành* (observability là đầu vào cho eval + governance loop)

Mỗi tín hiệu phải *trả lời một quyết định* và *đáng đồng tiền* — không đo để cho có.

Grafana dashboard live + Prometheus alerts + Jaeger distributed trace

- Grafana dashboard với 6 panels: RPS, P50/P95/P99 latency, error rate, GPU util
- 2 alert rules firing/resolving trong Alertmanager → Slack
- Distributed trace screenshot từ Jaeger cho 1 request end-to-end
- Evidently data drift report cho sample dataset

Cùng input $\rightarrow$ khác output. Test thường-thường (assert==) phá sản. Cần *distributional* eval thay vì pointwise.

Một request có thể tốn \$0.001 hoặc \$5 tùy prompt length + output. Token telemetry = *first-class metric* (xem §11).

Model không “crash” khi suy thoái — nó vẫn trả response, chỉ là sai/lệch. Cần baseline distribution + statistical tests (PSI, KL — §10).

Hallucinated tool args, infinite loops, context-window overflow, prompt injection — không có trong traditional service monitoring.

Hệ quả: traditional Golden Signals (latency/traffic/errors/saturation) chỉ phủ $\sim 50\%$ của AI failure surface.

Day	Artifact (từ track-2 trước đó)	Day 23 instrument cái gì
16	Cloud infra (Terraform EC2/EKS)	node_exporter + cAdvisor → Prometheus
17	Data pipeline (Airflow DAG)	DAG run duration, failure count via statsd_exporter
18	Data lakehouse (Spark/Delta)	Spark UI metrics → Prometheus, Delta commit rate
19	Vector & feature store (Qdrant/Feast)	Qdrant /metrics, query P99, embedding-distribution drift
20	Model serving (llama.cpp)	Tokens/sec, queue depth, GPU util (DCGM), TTFT/T-POT
22	Alignment evals (DPO model)	Eval-pass-rate as Prometheus gauge, human-thumbs-down rate

Day 23 = ngày tích

hợp. Mọi artifact từ Day 16–22 đổ telemetry về một stack duy nhất. Lab §5 sẽ wire toàn bộ chain.

RED (request): `inference_requests/errors_total`, `latency_seconds` · **USE** (resource): `DCGM_FI_DEV_GPU_UTIL`, `FB_USED`, queue depth. *Saturation* $\neq$ *util*: GPU 99% + queue 0 = healthy; + queue >50 = scale up.

- **Token throughput**: input+output tokens/sec
- **Hallucination/quality rate**: judge-LLM/regex
- **Output length dist**: histogram
tokens/response
- **Tool-call failure rate**: agents fail kiểu mới

200 0K vẫn có thể hallucinate · latency thấp vẫn có thể output rỗng · error 0% vẫn đốt \$5/req. **HTTP-200** $\neq$ **correctness** → eval-as-metric (\$10).

Định danh: “4th signal” = *model-behavior signals* — là metrics nhưng cần eval pipeline để tính.

Cardinality — Số lượng *unique label combinations* mà một metric có thể tạo ra. Mỗi combination = 1 time series. SaaS bills tính theo time series.

Lưu ý: Coinbase / Datadog 2022: \$65M annual bill, $\sim 13\times$ revenue per engineer. Nguyên nhân: custom-metric với label `user_id`. 100M users $\rightarrow$ 100M series *per metric*. tóm tắt ở §11.

Cardinality math:

`requests{user, endpoint, status}`
 $1M \text{ users} \times 50 \text{ endpoints} \times 4 \text{ status}$
 $= 200M \text{ time series cho một metric duy nhất.}$

Rule of thumb:

- $< 10K$ series / metric: OK
- 10K–100K: monitor
- $> 100K$: alert + redesign

Anti-label list: `user_id`, `request_id`, `prompt_hash`,
`session_id`, `traceparent`.

Tính năng	Trạng thái (06/2026)
Native histograms	GA từ v3.8.0 (28/11/2025); v4.0 mặc định on
OTLP receiver	Bật qua <code>--web.enable-otlp-receiver</code> ; <code>delta</code> = experimental
Remote-Write 2.0	Còn experimental (chọn qua <code>protobuf_message</code>); egress —50%
UTF-8 names	Mặc định; PromQL <code>{"http.req", job="api"}</code>
Agent mode	Stable, chờ riêng <code>--agent</code>
UI mới	Tree-view + Explain; UI cũ qua <code>old-ui</code>
Phiên bản mới nhất	v3.12.0 (28/05/2026)

Lưu ý: Breaking: range/lookback selector giờ **left-open, right-closed** $\Rightarrow$ `foo[5m]` luôn trả đúng 5 mẫu (trước đôi khi 6) — audit lại recording rule & alert. Nâng cấp lên v2.55 *trước* khi lên v3; rollback chỉ về v2.55.

Tiêu chí	VictoriaMetrics	Grafana Mimir 3.0	Thanos
Kiến trúc	1 binary / cluster nhỏ	Microservices + Kafka buffer	Sidecar + object store
Nén (vs Prom)	~10×; 0.4–0.8 B/sample	2–3× block storage	2–4× + downsampling
RAM	thấp (~1/5 Mimir)	cao (nhiều thành phần)	theo Prometheus
Multi-tenant	cơ bản	mạnh; per-tenant limits	qua external_labels
Quy mô	tiết kiệm, dễ vận hành	~1B series; 50M smpl/s	dán trên Prometheus

Stack mới + AI platform nhỏ → VictoriaMetrics. Đa tenant/-cluster + compliance → Mimir. Đã chạy Prometheus, muốn long-term rẻ → Thanos. **Cortex đã ngừng phát triển** (dùng Mimir).

Lưu ý: Downsampling *không* giảm dung lượng (thêm block); nó tăng tốc query dài. Mimir 3.0 (11/2025): Mimir Query Engine streaming + Kafka tách read/write.

Vì sao columnar thắng: Elasticsearch lưu mỗi field 3 lần (inverted index + `_source` + `doc_values`); columnar ghi mỗi cột 1 lần, codec riêng (Delta/Gorilla/ZSTD); Loki chỉ index label, lọc body vẫn full-scan.

OTel logs 50B	ClickHouse	ES
Dung lượng	2.43 TiB	12.01 TiB
Nén	16.3×	–
Query cold	26.2 s	111 s
Nạp 50B	<4 giờ	~5 ngày

Stack	Điểm khác biệt
ClickStack	ClickHouse+HyperDX, 1 image
SigNoz	APM OTel-native
Coroot	eBPF zero-config
OpenObserve	Rust, Parquet/S3
Uptrace	chỉ nhận OTLP

Cloudflare 600→60 B/doc; ContentSquare P99 nhanh 10×, rẻ 11× sau khi rời ES; rẻ hơn Datadog ~70–90%. Đánh đổi: full-text relevance vẫn là sân của Elasticsearch.

Request Rate · P50/P95/P99 Latency · Error Rate · GPU Util · Token Throughput · Cost/Hour. *Một dashboard = một câu hỏi; 6–9 panels max.*

Drift: embedding shift (KL) vs baseline, alert khi drift > ngưỡng. **Business:** cost/conversation, requests/user, top error types.

Grafonnet (Jsonnet) / **Terraform** Grafana provider → dashboard trong git, PR review, auto-deploy. Template vars: switch model/env/time. Manual dashboard = config drift.

Lưu ý: Panel-type: time-series (trend) · stat (current) · gauge (0–100%) · heatmap (distribution) · logs (LogQL).

Burn rate — Tốc độ ăn budget, chuẩn hoá so cho $1.0 = \text{on-track}$.

$14.4 =$ ăn hết budget 30 ngày trong ~ 2 ngày.

$1.0 =$ vừa đúng quota.

Recipe (canonical):

Alert nếu *cả hai* trong $\{5m, 1h\}$ window:

$\text{burn}_{5m} > 14.4 \wedge \text{burn}_{1h} > 14.4$

HOẶC cả hai trong $\{30m, 6h\}$:

$\text{burn}_{30m} > 6 \wedge \text{burn}_{6h} > 6$

SLO recap: SLI = metric đo được, SLO = target/window (vd 99.5%/30 ngày), SLA = SLO trong hợp đồng. **Error-budget**

policy: $>50\%$ deploy thoải mái · $25\text{--}50\%$ review · $0\text{--}25\%$ freeze · burned $\rightarrow$ rollback + P0.

Cửa sổ ngắn (5m) catch fast burn nhưng dễ flap.
Cửa sổ dài (1h) ổn định nhưng chậm. Yêu cầu *cả hai* fire $\Rightarrow$ no false positives, no missed real burns.

PromQL:

```
(error_rate_5m > 14.4 * 0.005)  
and (error_rate_1h > 14.4 * 0.005)
```

Single page = real outage. Không còn alert noise.

Semconv lỗi ở **v1.42**, nhưng toàn bộ `gen_ai.*` vẫn **Development** (đã tách repo `semantic-conventions-genai`).

Tín hiệu	Định danh chuẩn (status: Dev)
LLM call	<code>gen_ai.operation.name=chat</code>
Model	<code>gen_ai.request/response.model</code>
Token	<code>gen_ai.usage.input/output_tokens</code>
Provider	<code>gen_ai.provider.name</code> (đổi từ <code>gen_ai.system</code>)
Agent/Tool	<code>spans invoke_agent, execute_tool</code>
Nội dung	<code>gen_ai.input/output.messages</code>
Metric	<code>gen_ai.client.token.usage</code> (histogram)

- `gen_ai.system` → `gen_ai.provider.name`
- per-message *events* (v1.37) → 3 attribute gộp trên span
- content-capture **TẮT** mặc định (PII)

Lưu ý: Chưa “stable”. Khoá hành vi: `OTEL_SEMCONV_STABILITY_OPT_IN=gen_ai_latest` mặc định vẫn format cũ v1.36.

Agent (DaemonSet/sidecar): thu thập local, xử lý nhẹ.

Gateway (deployment): batch, tail-sampling, export.

Lưu ý: Tail-sampling cần mọi span của 1 trace tới cùng instance → route theo traceID qua loadbalancing exporter; single-tier âm thầm phá tail-sampling.

```
# Tier-1 agent: route by trace id
exporters:
  loadbalancing:
    routing_key: traceID
    resolver: { k8s: { service: otel-
                  gateway } }
# Tier-2 gateway: tail sampling
processors:
  tail_sampling:
    decision_wait: 30s
    num_traces: 50000
    policies:
      - name: errors
        type: status_code
        status_code: { status_codes: [ERROR
                                     ] }
```


DCGM_FI_DEV_GPU_UTIL — % thời gian GPU có ≥ 1 kernel resident — *không* đo bao nhiêu SM thật sự bận. Báo 100% dù chỉ 1/132 SM chạy (giống `nvidia-smi`).

Metric (DCGM)	Fleet thật	Ý nghĩa
DEV_GPU_UTIL	100%	có kernel chạy (báo cháy)
PROF_SM_ACTIVE	~35%	% chu kỳ có ≥ 1 warp/SM
PROF_PIPE_TENSOR_ACTIVE	~15%	Tensor Core thực sự bật (HMMA)
MFU (model FLOPs util)	25%	% compute đỉnh đạt được; “sự thật” duy nhất

Lưu ý: Alert: GPU_UTIL>90 and PROF_SM_ACTIVE*100<40 = kernel nhỏ / launch overhead / batch quá bé — “bận” giả. LLM serving tốt $\approx 35\text{--}45\%$ MFU; fleet trung vị <50% util, MFU trung bình **25.1%** (608 job H100, 2025); xAI Colossus $\approx 11\%$.

```
# 1) GPU "ban gia": util cao, SM that thap
DCGM_FI_DEV_GPU_UTIL > 90
    and DCGM_FI_PROF_SM_ACTIVE * 100 < 40
# 2) Throttle (VIOLATION counter us): rate>0
rate(DCGM_FI_DEV_THERMAL_VIOLATION[5m]) > 0
rate(DCGM_FI_DEV_POWER_VIOLATION[5m]) > 0
# 3) HBM sap het (<5% free) -> OOM
DCGM_FI_DEV_FB_FREE / (DCGM_FI_DEV_FB_USED
+ DCGM_FI_DEV_FB_FREE) < 0.05
# 4) XID fatal (48=DBE,79=off-bus,95=uncont)
changes(DCGM_FI_DEV_XID_ERRORS[5m]) > 0
# 5) Joule/token (FinOps)
increase(DCGM_FI_DEV_TOTAL_ENERGY_CONSUMPTION[1h])
/1000 / sum(increase(llm_output_tokens_total[1h]))
```

Nhóm

Metric

Bảo hòa	PROF_SM_ACTIVE
Tensor	PROF_PIPE_TENSOR_ACTIVE
Memory	DEV_FB_USED/_FREE
Power	DEV_POWER_USAGE
Nhiệt	DEV_GPU/MEM_TEMP
Throttle	*_VIOLATION
Lỗi	DEV_XID_ERRORS, ECC

Lưu ý: CLOCKS_EVENT_REASONS = bit-mask (PromQL không bit) → alert trên *_VIOLATION. Giám sát cả saturation & phần cứng.

```
# curl :8000/metrics (prefix "vllm:")
# Latency histograms (giây)
time_to_first_token_seconds      # TTFT
time_per_output_token_seconds    # ITL/TPOT
e2e_request_latency_seconds
request_queue_time_seconds
request_prefill_time_seconds     # V1
request_decode_time_seconds      # V1
# Throughput counters
prompt_tokens_total
generation_tokens_total
request_success_total{finished_reason}
# Scheduler / KV gauges
num_requests_running
num_requests_waiting
kv_cache_usage_perc             # 0..1 fraction (V1)
num_preemptions_total          # = RECOMPUTE
prefix_cache_queries_total
prefix_cache_hits_total
```

- gpu_cache_usage_perc → kv_cache_usage_perc
- num_requests_swapped **bỏ** (V1 không swap CPU)
- Preempt = **RECOMPUTE** + prefix-cache, không SWAP
- Prefix-cache giờ là **counters** (_total) → tự tính hit-rate bằng PromQL

Lưu ý: Lên V1: dashboard/alert cũ dựa trên gpu_cache_usage_perc sẽ **gây im lặng**. Hồi sinh tạm: `--show-hidden-metrics-for-version=0.7`.

Hai pha prefill/decode sinh ra bốn SLI riêng:

- **TTFT**: tới token đầu (network+queue+prefill) — SLI tương tác chính.
- **TPOT** = $(E2E - TTFT) / (N_{out} - 1)$ — tốc độ decode/request.
- **ITL**: khoảng cách 2 token liên tiếp — độ “mượt” của stream (alert p99).
- **Goodput**: req/s đạt cả SLO TTFT và TPOT (khác throughput thô).

Use case	TTFT p99	ITL p99
Voice agent	150 ms	30 ms
Code completion (inline)	100 ms	25 ms
Interactive chat	300 ms	50 ms
RAG chat	400 ms	80 ms
Batch / async agent	3000 ms	200 ms

Goodput — SLO-attained throughput — chỉ đếm request đạt CẢ TTFT lẫn TPOT. NVIDIA **AIPerf** (kế nhiệm GenAI-Perf) đo bằng cờ `--goodput "time_to_first_token:100 inter_token_latency:10"`.

PD Aggregation — Sarathi-Serve

Continuous batching + chunked-prefill với token budget cố định; prefill chen cùng batch decode (stall-free).

- ☒ TTFT tốt, throughput cao.
- ☐ Prefill chen decode $\Rightarrow$ TPOT tail xấu khi tải cao.

Số: $2.6\times$ (Mistral-7B, A100) tới $5.6\times$ (Falcon-180B).
Tight-TTFT/relaxed-TPOT: **97%** SLO.

PD Disaggregation — DistServe / Dynamo / llm-d

Prefill và decode tách lên GPU pool riêng; lộ *metric mới*: KV-transfer time, prefill-queue, overlap score (KV-routing).

- ☒ Tight-TPOT (60ms): **98%** vs 7% (gộp).
- ☐ Năng lực prefill thấp hơn $\Rightarrow$ TTFT dễ nghẽn.

Số: DistServe $4.48\times$ goodput; llm-d KV-routing P90 TTFT 0.54s vs 94.9s (cache-blind).

```
# Scale on QUEUE DEPTH, not GPU%:  
# GPU% is ambiguous (efficient vs saturated)  
.  
apiVersion: keda.sh/v1alpha1  
kind: ScaledObject  
spec:  
  scaleTargetRef: { name: llama-3b-vllm }  
  minReplicaCount: 1  
  maxReplicaCount: 8  
  pollingInterval: 5           # seconds  
  triggers:  
  - type: prometheus  
    metadata:  
      query: sum(vllm:num_requests_waiting)  
      threshold: "2"           # queue>2 ->  
                               scale
```

- **Gateway API Inference Extension:**
InferencePool chấm “saturation” =
queue + KV util
- **llm-d** (CNCF Sandbox, ~v0.8): EPP
route theo queue + prefix-locality
- **Dynamo 1.0** (GA 2026):
router_kv_hit_rate, label
worker_type

Lưu ý: Tín hiệu phụ
vllm:kv_cache_usage_perc báo cạn
KV trước khi queue dâng. Red Hat/Open-
Shift AI: queue depth là trigger chính.

1.0 — Ba trụ cột

- Metrics + logs + traces tách rời, **nhiều nguồn sự thật**
- Tổng hợp tại **write-time**: phải chọn trước câu hỏi sẽ hỏi
- Chi phí tăng theo **cardinality**; không dám gán `user_id` làm label
- “Metric là aggregate **không thể un-aggregate**”: chỉ còn số tổng + exemplar

2.0 — Wide events

- **Một** nguồn sự thật: structured event siêu rộng (100–1000 field/event)
- Tổng hợp tại **read-time**; giữ raw event để query ad-hoc
- Lưu dạng cột (Parquet/Arrow); cardinality **vô hạn**, không phạt
- Suy ra metric/log/trace **động** từ cùng một event

Grafana **Beyla** đã hiến tặng cho OpenTelemetry → **OBI** (OpenTelemetry eBPF Instrumentation), vẫn *Development* (v0.x, mục tiêu GA 1.0 cuối 2026). **Alloy** (OTel-native) thay Grafana Agent + Promtail (**Promtail EOL 03/2026**):

```
beyla.ebpf "default" {
  open_port = "80,443,8080-8089"
  output {
    traces  = [otelcol.processor.batch.in]
    metrics = [prometheus.remote_write.in]
  }
}
// K8s DaemonSet needs: hostPID=true,
// CAP_NET_ADMIN, mount /sys/fs/cgroup
// Network traceparent => kernel 5.17+
```

Hybrid — eBPF lo RED/latency/service-graph baseline (0 code, mọi ngôn ngữ); SDK lo span sâu + nghiệp vụ. `span.metrics.skip=true` tránh metric trùng.

Continuous profiling — Liên tục lấy mẫu stack-trace CPU/heap/lock trên production, quy về từng dòng code — trả lời “dòng nào đốt CPU/RAM/\$”.

Tín hiệu	Câu hỏi	OTel
Metrics	Có lỗi không?	GA
Traces	Ở đâu?	GA
Logs	Chi tiết?	GA
Profiles	Dòng nào?	Alpha

- ☒ eBPF: **không sửa code**, không restart
- ☒ Overhead ~1% CPU, <250 MB RAM
- ☒ Pyroscope 2.0 (04/2026): diskless, object-store, —95% symbol storage
- ☐ OTel Profiles **Alpha** (05/2026): chưa dùng cho critical prod

Metric/trace nói *cái gì* chậm; profile chỉ thẳng *dòng code* — whole-system, gồm cả kernel & thư viện. Span profiles gắn flame graph vào đúng span chậm: từ “service chậm” → “dòng 142 trong tokenizer”.

Loại drift	Định nghĩa hình thức	Cần GT?	Detector điển hình
Data	$P(X)$ shift (covariate); $P(Y X)$ ổn định	Không	PSI / KS / MMD per-feature
Label	$P(Y)$ shift (prior); $P(X Y)$ ổn định	Có	Chi-square trên class proportions
Concept	$P(Y X)$ shift — relationship đổi (hardest)	Không (CBPE)	NannyML CBPE/PAPE estimate metric vs SLO
Prediction	$P(\hat{Y})$ shift	Không	KL divergence trên $\hat{Y}$ histograms
Embedding	embedding distribution shifts	Không	Cosine-to-centroid, MMD, model-based ROC-AUC
Prompt	LLM behavior shifts <i>không</i> edit prompt	Không	Eval pass-rate per prompt-version × model
Model	umbrella: performance vs baseline degrades	Tùy	All of the above; PAPE preferred (no labels)
Agent	behavioral signature shifts (tool-call freq, plan length)	Không	Behavioral signature + MAST failure-class rate

Lưu ý: Concept drift = hardest: CBPE/PAPE (NannyML) estimate *performance* không cần labels. Agent drift: behavioral signature + MAST (Cemri et al., NeurIPS 2025).

```
# Langfuse SDK v4 (OTel) online eval
@observe()
def answer(q):
    ctx = retrieve(q)
    out = generate(q, ctx)
    flagged = guardrail(out)          # fast
    # 100% on flagged, else 10% sample
    if flagged or random() < 0.10:
        s = judge_faithful(out, ctx)
        langfuse.score_current_span(
            name="faithfulness", value=s)
    return out
```

sample (5–20% + 100% flagged) →
judge/RAGAS → Prometheus gauge →
alert nếu drop >10% → failing trace vào
annotation queue (HITL) → dataset. **SLI**
mới: “% pass eval ≥ 0.7 ”.

Cost budget — Judge <10–15%
prod LLM cost; >25% → distill/giảm sample.
Evals dùng lại từ Day 22 (training → prod).

Faithfulness / Groundedness

$$= \frac{|\text{claims supported by ctx}|}{|\text{total claims in answer}|}$$

Answer Relevancy

$$= \text{mean cos}(\vec{q}, \vec{q'_i}), q'_i \text{ là câu hỏi sinh ngược từ answer}$$

Context Precision

rank-weighted: chunk relevant ở hạng cao $\Rightarrow$ điểm cao

Context Recall

$$= \frac{|\text{info cần, có trong ctx}|}{|\text{info cần để trả lời}|}$$

Multi-signal groundedness — NLI trên từng claim: **contradiction** = **hallucination**, neutral = unsupported, entail = grounded; cộng citation-validity. Online: chạy trên context đã retrieve, *không* cần ground-truth answer $\Rightarrow$ deploy được như SLI.

4 cost drivers — (1) unique time series, (2) log GB/ngày, (3) trace span count, (4) retention. Hidden killer: *cardinality* thổi (1).

Worked example: $100 \text{ svc} \times 50 \text{ metric} \times 20 \text{ label} = 100\text{K}$ series; Datadog $\$0.05/\text{series} \Rightarrow \$5\text{K}/\text{th} + 200\text{GB}/\text{ngày} \log \times \$2/\text{GB} \Rightarrow \$12\text{K}/\text{th} = \sim \$17\text{K}/\text{th}$ (chưa traces). *Grafana 2025: obs = 17% chi tiêu compute (median 10%); 37% thấy “quá đắt”; hoá đơn +30–50%/năm (+40–200% khi thêm LLM telemetry). Coinbase \$65M Datadog (2021–22) là cảnh báo lịch sử (§3).*

- **Sampling** 5% + 100% lỗi/chậm
→ –80–90% trace
- **Drop cardinality** (bỏ `user_id`); Adaptive Metrics $\approx -35\%$
- **Tiered retention** hot/cold/archive
→ –30–50%
- Recording rules pre-aggregate; drop dev/staging
- Pipeline (Cribl/OTel Collector) lọc trước backend

Lưu ý: 20–50% metrics không bao giờ được query — vẫn trả tiền. Đặt **cardinality budget** + audit hàng tuần.

Platform	OSS / Self-host	OTel	Pricing axis	Eval
Langfuse v4	MIT, full self-host	✓	event-based	✓ judge
Arize Phoenix	OSS, self-host	✓	free; AX=spans/mo	✓
Comet Opik	Apache-2.0, self-host	✓	free OSS	✓ judge
Helicone	Apache-2.0, self-host	✓	maintenance (Mintlify)	partial
LangSmith	closed; ENT self-host	✓	seat + traces	✓
Braintrust	closed; ENT self-host	✓	free 1M spans	✓
W&B Weave	closed; self-managed	✓	usage-based	✓
Datadog LLM Obs	closed SaaS only	✓	~\$8/10k req	✓

OSS (MIT, ClickHouse-backed), OTel SDK v4, multi-tenant. Bonus BONUS-llm-native-obs/ wire LangChain → Langfuse.

Bắt buộc self-host OSS → Langfuse / Phoenix / Opik. OTLP span ⇒ đổi backend không re-instrument. **VN PDPL 91/2025** (data-sovereignty): self-host hoặc SaaS có VN region (AWS Singapore / GCP asia-southeast1).

Thay đổi “an toàn” + giới hạn cứng ẩn: AWS US-EAST-1 DNS race (~15h, ~1000 dịch vụ); Azure AFD config; Cloudflare 200-feature cap → FL2 panic. **Lesson:** đo *bán kính lan*, không chỉ “up”.

\$50K/24h. Chatbot hallucinated promo “WELCOME50”. **Lesson:** eval-as-metric (§10); cap monetary actions.

Apr 2025. no deployment evals; A/B thiếu signal. **Lesson:** eval breadth $\neq$ count (§15).

2024→25. Mean dashboards giấu variance; low-conf degrade âm thầm. **Lesson:** track outcome *distribution*.

Jul 2025. 1206 records dropped; no env-isolation/SQL allowlist. **Lesson:** agent-obs = product safety (§13).

Focus systems & decisions, không cá nhân. Template: Timeline · Impact · Root-cause · Action-items · Lessons. Escalation 0/5/15/30m.

Pattern: monitoring *tồn tại* nhưng *thiếu signal đúng*: eval-as-metric / deployment-eval / outcome-distribution / agent-audit / blast-radius. Mapping §2 §6 §10 §13 §15.

Luật 91/2025/QH15 ban hành 26/06/2025, **hiệu lực 01/01/2026**; Nghị định 356/2025 hướng dẫn. Thay thế Nghị định 13/2023 (PDPD).

- **Báo cáo vi phạm trong 72h kể từ khi *phát hiện*** (trước đây tính từ lúc xảy ra) — log/trace phải đủ để dựng lại timeline sự cố.
- **Mức phạt:** tới 5% doanh thu năm trước (chuyển dữ liệu xuyên biên giới); tới 10× doanh thu từ mua/bán dữ liệu trái phép.
- Log thường chứa PII (IP, user_id, prompt LLM) → **redaction tại pipeline** (OTTL §7), retention có chủ đích, data residency.

Lưu ý: Logs/traces giờ là *dữ liệu cá nhân* chịu điều chỉnh: “log mọi thứ mãi mãi” vừa đội chi phí (FinOps §11) vừa tạo rủi ro pháp lý.

Agent vs LLM-app — Một LLM-app làm *một* request → một response. Một agent làm *N bước*: plan → tool-call → observe → repeat. Thất bại có thể nằm ở bất kỳ bước nào — tracing per-call không đủ, phải xem toàn trajectory.

Trace metric vs outcome metric:

Trace Mỗi step success/fail

Outcome Task hoàn thành đúng?

Trace 100% pass nhưng outcome wrong: agent gọi đúng API, lấy đúng data, nhưng *trả lời sai user*.

Lưu ý: Soft success problem: tool trả 200 OK, JSON valid, *nhưng* content rác. Trace metric không catch. Cần judge-LLM grade trên *trajectory* đầy đủ, không chỉ final answer.

- **AgentOps:** Python SDK, session replay
- **Phoenix agent traces:** OTel-native, OpenInference
- **LangGraph Studio:** graph-replay UI
- **Langfuse v4:** OSS self-host, ClickHouse-backed, OTel SDK

- `gen_ai.operation.name` — 5 verb loop:
`create_agent` / `invoke_agent` / `plan` /
`invoke_workflow` / `execute_tool` (frame sau)
- `gen_ai.agent.id`, `gen_ai.tool.name/.call.id`
- `gen_ai.usage.input/output_tokens`,
`cache_read`
- Phân loop qua `span.kind`: `CLIENT` (remote) vs
`INTERNAL` (in-loop)

- Span *KIND* tường minh: `AGENT/-`
`TOOL/LLM/RETRIEVER/GUARDRAIL...`
- `tool.name`, `llm.token_count.*`
- Native trong Phoenix; có trước OTel GenAI

Hội tụ: nhiều SDK *dual-emit* cả hai. Cây span MCP: frame “Span Tree”. Tool stack (AgentOps/Phoenix/Langfuse v4/LangGraph Studio): frame mở đầu §13.

```
invoke_agent support-router      [INTERNAL]
|- chat gpt-4o                  [CLIENT]
|   gen_ai.usage.input_tokens = 1523
|   finish_reasons = [tool_calls]
|- tools/call query-orders      [CLIENT] # MCP client
|   mcp.method.name = tools/call
|   mcp.session.id = sess-abc
|   gen_ai.tool.name = query-orders
|   `-- tools/call query-orders [SERVER] # MCP server
|       (joined via W3C trace-context)
`- chat gpt-4o                  [CLIENT]
    gen_ai.usage.cache_read.input_tokens = 1523
    finish_reasons = [stop]
```

Mỗi MCP `tools/call` = *cặp* span (client + server) nối bằng trace-context, *không* phải 1 span. Nếu bật cả GenAI + MCP instrumentation, MCP *làm giàu* span `execute_tool` sẵn có thay vì nhân đôi. Status: semconv v1.42 (05/2026), vẫn *Development*.

```
# gen_ai.operation.name biến VÒNG LẠP thành span hàng nhất (5 verb):  
#   create_agent | invoke_agent | plan | invoke_workflow | execute_tool  
# SPAN.KIND phân biệt loop local vs remote:  
invoke_agent router      [CLIENT]   # agent-service từ xa (A2A/Bedrock)  
invoke_agent react-loop  [INTERNAL]  # ReAct loop trong-framework  
plan          decompose   [INTERNAL]  
execute_tool  query-orders [CLIENT]   # span-name = "execute_tool {tool}"  
# Metric REQUIRED duy nhất: gen_ai.client.operation.duration  
#   -> KHÔNG có metric so-vòng chuẩn: DERIVE = count(execute_tool span)
```

cache_read/cache_creation.input_tokens (prefix-cache reuse tăng theo turn = loop khỏe), reasoning.output_tokens, gen_ai.conversation.id = khóa nổi phiên long-horizon.

Lưu ý: Vẫn *Development*, từ v1.42 gen_ai.* dời sang repo semantic-conventions-genai, bật bằng OTEL_SEMCONV_STABILITY_OPT_IN = gen_ai_latest_experimental.

```
# p95 tool latency per tool (RED via spanmetrics)
histogram_quantile(0.95,
  sum by (le, tool) (rate(
    tool_call_duration_seconds_bucket[5m])))

# tool error rate
sum(rate(tool_calls_total{status="error"}[5m]))
  / sum(rate(tool_calls_total[5m]))

# loop/runaway rate: runs over step cap
sum(rate(agent_runs_total{steps="gt15"}[1h]))
  / sum(rate(agent_runs_total[1h]))

# cost per SUCCESSFUL task (USD), not per LLM-call
sum(rate(agent_cost_usd_total[1h]))
  / sum(rate(
    agent_tasks_total{outcome="success"}[1h]))
```

Loop guard: alert khi 1 run > 15 steps; hash tool-call so với rolling 5-call window (dup = no-progress). Agents đốt ~50× token so với chatbot; 1 runaway session từng tốn \$12k.

Stack agent 2026 = 3 lớp —

- **Prompt** — chỉ dẫn (cần nhưng *không còn đủ*)
- **Context** — model thấy gì ở mỗi bước
- **Harness** — phần mềm *tất định* bao quanh model: tool-orchestration, state, error-recovery, verification loop

Harness giờ là **đòn bẩy độ-tin-cậy chính**, không phải prompt (Anthropic, *Building Effective Agents*).

Model = CPU · Context = RAM · **Harness = OS**
· Agent = App.

Quan sát harness = quan sát OS (scheduler, I/O, retry), *không* benchmark CPU. Token/latency chỉ đo CPU — cái còn thiếu là **EKG của vòng lặp**.

Lưu ý: **Durability** lộ ra *sau* tool-call thứ 50–100 — nơi engineering-churn dồn vào (harness refactor liên tục).

1

Hỏi 1 câu ở MỖI lớp: “tín hiệu nào cho biết lớp này khỏe?” Phần lớn dashboard chỉ trả lời cho lớp model.

```
# Claude Code: bat OTel, KHÔNG tu phat minh metric
export CLAUDE_CODE_ENABLE_TELEMETRY=1      # -> OTLP
# claude_code.token.usage {type=input|output|cacheRead|cacheCreation, model}
# claude_code.cost.usage  USD {agent.name, skill.name, plugin.name}
# claude_code.code_edit_tool.decision {accept|reject}
# claude_code.active_time.total          (metric 60s / log 5s)
# Span tree = anh giao khoa cua loop:
claude_code.interaction
|- llm_request          gen_ai.request.attempt=2    # retry = span-event
|- tool.blocked_on_user decision=accept            # HITL-wait -> SLO
|- tool.execution       subagent (summary ~1-2K tok)
# Codex CLI: namespace RIENG codex.* (chua align gen_ai.*)
# codex.tool_result {outcome,duration} | codex.sse_event {tok in/out/cached}
# BAY: codex_exec headless phat log+trace NHUNG khong metrics (issue #12913)
```

Map vào RED/USE đã học: blocked_on_user = HITL-wait đặt SLO được; gen_ai.request.attempt = retry/backoff; sub-agent trả summary ~1-2K token = pattern cô lập context *quan sát được*. OTel GenAI semconv vẫn *Development* (06/2026).

- Claude Code: nén $\sim 167\text{K}/200\text{K}$ ($\approx 83\%$, buffer $\sim 13\text{K}$); **auto-compact mặc định $\sim 95\%$**
- Letta $\sim 90\%$ usage · Pi chưa reserve 16,384 tok · OpenClaw nén ở 50% history (+ TTL prune)

Lưu ý: Không có metric “**compaction event**” sẵn — tự đếm bằng hook PreCompact/PostCompact, log token pre/post.

Cascade nén (rẻ-trước) —
budget-reduction $\rightarrow$ snip $\rightarrow$ microcompact $\rightarrow$
context-collapse $\rightarrow$ auto-compact.

cap đọc file/tool $\sim 25\text{K}$ tok; *tool-result clearing* mặc định trigger ở **100K** input-token, giữ 3 tool-use. “Code-exec w/ MCP”: 1 ví dụ Anthropic $\sim 150\text{K} \rightarrow 2\text{K}$ tok.



Context rot: recall giảm khi token tăng (attention n^2). Compaction là last-resort: summarize rồi re-init, giữ kiến trúc/bug chưa giải, bỏ tool-output dư.


```
# Policy harness check TRUOC moi buoc (vua control vua observable)
limits:
  max_steps: 100          # OpenHands max_iterations
  max_usd: 3.00           # SWE-agent per_instance_cost_limit
  max_seconds: 1800
budget_pressure:          # chen prompt theo % -> observable
  at_70pct: "con N vong"
  at_90pct: "PHAI tra loi ngay"
# stop_reason distribution = KPI suc khoe deploy:
# done | budget_exhausted | error | loop_guard
# Don vi quan sat = TASK, khong phai CALL:
# gan task_id -> gop 5 span/luot thanh 1 dong task
# cost_per_task = sum(cost) / count(task outcome=success)
```

Xu hướng 2026: USD là budget chính cascade user→project→mission→thread; iterations/token/wall-clock chỉ là backstop. stop_reason lệch về budget/error = cảnh báo sớm. Reuse agent-SLI PromQL (§13) roll-up theo task_id.

- **Stuck:** cùng (tool, args_hash) ≥ 3 lần trong rolling window $\rightarrow$ thoát sớm kèm chẩn đoán
- **Oscillation:** sửa A lại tái sinh B ($A \leftrightarrow B$ flap), không hội tụ

Loop khỏe — commit tăng dần · E2E test pass
· code gọn · artifact on-disk (progress.txt, git) làm state-recovery.

Lưu ý: Chữ ký thoái hóa (Anthropic): “one-shotting” cả app · cạn context giữa chừng · khai “done” khi mới xong một phần.

qua 3–8 checkpoint: *structural erosion* ($CC > 10$)
tăng ở ~ 77 –80% trajectory; verbosity +75–90%;
agent tốt nhất pass chỉ 14.8% checkpoint.



Mỗi chữ ký = 1 metric: context-util spike, commit-cadence flatline, complexity/verbosity leo theo iteration, span task_complete quá sớm. Long-horizon là *metric-set*, không phải 1 error đơn.

METR “task time horizon” — Độ dài tác vụ (phút-người) mà agent xong ở **50% reliability**. “Moore’s Law cho agent”: doubling ~ 196 ngày (2019–25), tăng tốc ~ 89 ngày từ 2024.

Lưu ý: 50% là **press-release**. 80%-horizon (deploy được) nằm xa bên *trái*: chọn reliability cao $\rightarrow$ autonomy-duration dùng-được *sụp*. Delta đó = error-budget tác vụ dài.

Frontier (METR, 01/2026): Opus 4.5 ~ 320 phút (~ 5.3 h) ·
GPT-5 ~ 214 · o3 ~ 121 · Opus 4 ~ 101 . Tác vụ > 16 h
“unreliable” — *eval-harness bão hòa trước agent*.

Anthropic: P99.9 turn-duration $< 25 \rightarrow > 45$ phút (10/25 $\rightarrow$ 01/26), median ~ 45 s. Đuôi tự-hành trình = oversight-cost tăng.



Nếu agent chạy nhiều giờ không người, observability của bạn cũng phải. “Autonomy duration” vừa là capability vừa là risk-dial cần bound.

per-step hazard $\approx$ hằng $\rightarrow$ success giảm *cấp số nhân* theo độ dài (98%/bước $\rightarrow$ $\sim$ 33% fail qua 20 bước).
Caveat: hazard thực *giảm dần* (sống sót sớm = tự ổn định). (Ord, 2025)

Taxonomy **HORIZON** — 72.5% lỗi *process-level* (planning, catastrophic forgetting) vs 27.5% design-level. **Scale model KHÔNG fix** — phải observe + kiến trúc memory.

Lưu ý: 3 trục thất bại **RIÊNG**, đều nổ **TRƯỚC** khi token đầy (alarm “context full” bỏ sót): *context rot* (độ-dài-input) · *goal drift* (attention kéo về tool-output mới) · *compounding error* (số-bước).

goal-adherence · context signal/noise · plan-deviation rate · compaction_count + token pre/post. Đây là *drift detector*, không phải token gauge.

```
# REUSE agent-SLI PromQL (section 13) làm MINING QUERY:
# sub-trajectory nào đang được PROMOTE thành skill?
topk(10, sum by (tool_seq_hash) (
  count_over_time(agent_subtraj_total{
    verified="true"}[7d])           # tại xuất nhiều lần
)) and on() (avg by (tool_seq_hash)(
  agent_subtraj_cost_usd) > 0.05) # ton kém → đang đang gọi
# Rubric: frequency x cost x verifiability
#   mine SUCCESS cluster → skill / tool mới
#   mine FAILURE cluster → eval-case / guardrail
```

Voyager (2023): chỉ commit program đã *self-verify* vào skill-lib → $3.3 \times$ item, tech-tree $15.3 \times$. **Trace2Skill** (2026): trace → success/error-analyst → merge thành SKILL.md.

Lưu ý: Skill-lib **không append-only**: cần GC/promote/demote như retention+cardinality TSDB. Deploy = SKILL.md progressive disclosure (name→full→files).

Trace của bạn là training-set — GEPA

(ICLR 2026): đọc trajectory, reflect bằng tiếng Anh, tiến hóa prompt theo Pareto → vượt GRPO $\sim 6\text{--}20\text{pp}$ với ít hơn tới $35\times$ rollout; chạy được với ~ 10 ví dụ.

ACE: Generator/Reflector/Curator cập nhật “playbook” bằng *delta* (chống context-collapse), +10.6% agent, không cần label.

Nubank (KDD 2026): GEPA-trong-DSPy nâng judge-accuracy 69→89%, Cohen κ 0.00→0.75. **Decagon:** 20–100 ví dụ *thắng* 500 (500 gây prompt-bloat). Reflector **phải** là model frontier.

Lưu ý: Đây là vòng **MAPE-K** (Monitor-Analyze-Plan-Execute + Knowledge) — nổi lại nền SRE/AIOps. NVIDIA Data-Flywheel distill 70B→1B cắt >98% chi phí.



Observability không chỉ *sửa* prompt — nó *chọn* được model rẻ hơn.
Trace→dataset→experiment→deploy giờ one-click (Braintrust Loop, Arize Alyx).

Lưu ý: **Goodhart:** tối ưu mù vào metric có thể *giảm* chất lượng thật. “When Better Prompts Hurt” (2026): prompt “best-practice” đề constraint task-specific → extraction 100→90%, RAG-citation 93→80% (dù instruction-following +13pp).

Offline = gate, reproducible, pre-deploy. **Online** = sense, async no-latency, score-as-deploy-gate. Held-out validation chống over-fit 1 judge.

Who watches the judge? — LLM-judge cũng *drift*: version hóa dataset/rubric/prompt/threshold/*judge-model* + calibrate định kỳ với nhãn người. Judge chỉ có nghĩa nếu *tương quan* với người.

Lưu ý: **Vendor freshness:** Helicone vào maintenance sau khi Mintlify mua (03/2026) — đừng dạy tool chết. OSS (Phoenix/Langfuse) vs SaaS (Braintrust/LangSmith/Datadog).



Vòng tự-cải-thiện chỉ tốt bằng eval của nó. Tách offline-gate vs online-sense; mọi prompt-change phải qua eval-suite *task-specific*, không chỉ điểm tổng.

Test	Khi dùng	Math hint / caveat
Welch's t-test	Means, unequal variance/N (almost always production)	$t = (\bar{X}_1 - \bar{X}_2) / \sqrt{s_1^2/n_1 + s_2^2/n_2}$
Mann-Whitney U	Heavy-tailed (revenue, latency)	Tests stochastic dominance, không median — mất magnitude info
Wilson score CI	Proportions, small N hoặc extreme rates (0/1)	Shrinks toward 0.5 với N nhỏ — regularization đúng
mSPRT (always-valid)	Sequential testing, peek-friendly	Optimizely (since 2015); peek 10× raises FPR 5%→40%
CUPED	Covariate-adjusted variance reduction (pre-experiment data)	Bing 50% variance cut = half sample. Spotify/Eppo ship it
Bayesian binomial	Conversions, “ship if $P(\text{lift} > 0) > 0.95$ ”	GrowthBook default; per-test choice

Sample size scales với *square* của precision (detect half effect = $4 \times$ sample). 5% baseline, 1% MDE, 80% power $\Rightarrow \sim 31K/\text{arm}$.

Lưu ý: **MAB** $\neq$ **A/B**. Use khi ≥ 5 variants hoặc fast iteration. Achieves $O(\ln T/\Delta)$ regret (Thompson sampling). DoorDash dispatch.

Pattern	Mô tả	Pros	Cons / Cite
Shadow / mirror	New model nhận 100% prod traf- fic, predictions <i>discarded</i>	Zero user impact	2× infra. Uber 2025: 75%→100%
Champion / challenger	1 baseline + N challengers, weighted traffic, promote on cri- terion	Continuous im- prove	Multi-model deploy ops
Interleaving	Same query → both arms, merged + attribution	10×–100× sample-eff	Chỉ rankings/lists. Netflix
Blue / green Canary	Full swap, instant rollback button Graduated 1%→5%→25%→100% với auto-rollback	Simplest rollback Bounded blast ra- dius	Big-bang, no gradual Longer to 100%
Switchback	Randomize bằng (geo × time- window) units	Handles network effects	Power complications. Do- orDash

Uber shadow >75%→100% critical ML (2025); **Airbnb** interleaving 0.5% A/B-time & 4% traffic = same conclusion; **Thumbtack** 400 vs 40K samples (100×); **LinkedIn** 41K concurrent tests / 700M members; **Spotify** 750M users, 10K+ exp/yr (Apache-2.0); **Grab** (SEA) geo-temporal.

Các §1–14 dạy *quan sát* (hệ + agent); từ **§15** ta chuyển sang *vận hành cả vòng đời*. Observability đã là *table-stakes* — mặt trận chưa giải quyết là **eval + governance**, đúng phần các kỷ luật Ops sở hữu.

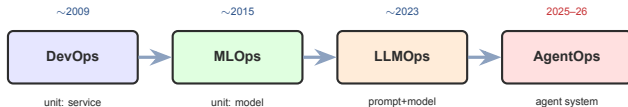
Đội có agent prod	%
Có observability	94%
Full tracing	71.5%
Offline evals	52.4%
Online evals	37.3%

LangChain SoAE (KS 11–12/2025, n=1.340).



Quan sát là điều kiện cần; **eval + governance + lifecycle** mới đủ để đưa ML/LLM/agent vào sản xuất.

Bước tiếp của độ trưởng thành là *eval-as-gate + governance*, không phải thêm panel. Trinity: §16→§19.



Mỗi thể hệ *kế thừa*, không thay thế: **DevOps** vận hành

code tất định; **MLOps** thêm model xác suất (drift §10, registry §17); **LLMOps** thêm prompt+context+RAG; **AgentOps** thêm vòng lặp + tool tự hành. “Compound AI systems” (Berkeley BAIR, 02/2024) là bước ngụ ý model → *hệ thống*.

Lưu ý: Thuật ngữ “trinity” / GenAIOps / AgentOps là *framing cộng đồng + vendor*, **chưa** chuẩn hoá ngành — dùng để tư duy, đừng tranh cãi nhần.

Bước	Cái gì xảy ra	Output / Gate	Obs
1 Data val	schema + drift vs baseline	quarantine nếu drift	§10
2 Train	run reproducible, log params	run lineage	§7
3 Eval gate	metric vs threshold + regression	fail nếu < SLO	Day 22
4 Registry	version + alias @challenger	pending approval	—
5 Deploy	canary / shadow rollout	auto-rollback	§15
6 Monitor	live metric + drift detect	alert / burn-rate	§6/§10
7 Retrain	trigger chạy lại pipeline	candidate mới	§10

CI = test code+data · **CD** = ship model · **CT** = *Continuous Training*: retrain khi có trigger. CT là vòng *không* có trong software thuần. Quy mô thật: một hãng lớn chạy >20K training job/tháng, 5K+ model ở production.

CACE — **Changing Anything Changes Everything.** Đổi 1 feature/data/hyperparam $\Rightarrow$ phân phối output dịch toàn hệ. Không có “isolated change” trong ML.

Phần ML code rất nhỏ; còn lại là config, data collection, serving, monitoring.

- **Glue code** dán quanh model
- **Pipeline jungles** scrape/join chồng chất
- **Config debt** hàng trăm flag không test
- **Undeclared consumers** đọc lên output
- **Feedback loops** model tác động input của nó (§10)

2026 vẫn là framing chuẩn — nghiên cứu SATD cho phần mềm LLM (arXiv 01/2026) tái khẳng định glue/config debt trội hơn ML code.

Mức		Tự động hóa	Quan sát khớp ở đâu
Google Manual	L0	notebook → handoff; không CI/CD ⇒ train-serving skew	uptime cơ bản
Google L1 ML pipeline + CT	ML	data+model validation, feature store, metadata store	drift + perf (§10)
Google L2 CI/CD pipeline	L2	CI/CD cho <i>chính pipeline code</i>	end-to-end + lineage (§7)

Lưu ý: Đừng nhầm 2 mô hình: **Google** = 3 mức (0/1/2, chuẩn để dạy CT). **Microsoft Azure** = 5 mức (0–4): CI/CD chỉ xuất hiện ở **L3**, auto-retrain theo drift ở **L4**; GenAIOps được đo *riêng*. Hãy nói rõ bạn dùng mô hình nào. Mức cao = tốc độ tạo model mới khi data/code đổi.

- **On-demand** (thủ công)
- **Scheduled** (cron — domain ổn định)
- **New data** (batch mới landing, Day 17)
- **Decay / drift** → §10 (chi tiết PSI/KS ở §10)

BP 2026: schedule *baseline* + threshold-trigger (hybrid). CD4ML: version code+data+model.

thumbs-down↑ · retry/regenerate↑ · response-length shift · cost-per-task↑ (model struggling) — leading indicators *trước* khi metric tụt.

Lưu ý: Safeguards: candidate phải beat baseline trên eval-gate (Day 22) + canary 5% (§15) + cooldown (max 1 retrain/tuần) + human-approval để promote. “Auto-retrain” không eval-gate = tự đẩy model tẹ ra prod.

```
import mlflow                                # MLflow 3 (GA 06/25)
mlflow.set_experiment("fraud")
# LoggedModel = first-class (model<->run<->
#   trace<->prompt<->eval)
with mlflow.start_run():
    mlflow.sklearn.log_model(clf, name="fraud")
mlflow.genai.evaluate(data=ds, scorers=[s]) # OSS
# promote/rollback by ALIAS (stages dep. 2.9):
client.set_registered_model_alias(
    "fraud", "champion", version=3)
# load -> models:/fraud@champion
```

Stages (None/Staging/Prod) *deprecated* từ 2.9 → alias @champion/@challenger + tags. **Alias swap = promote/rollback** (§15 shadow/canary, Day 12). Databricks: databricks-uc (auto lineage).

Data (DVC/lakeFS, Day 18) + Model (registry/seed) + Code/env (git+container). Trace/eval §7/§10.


```
name: ml-ci-gate
on: [pull_request]
jobs:
  gate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: pip install -r requirements.txt
      - run: pytest tests/test_data.py # schema+drift
      - run: python train.py --out model.pkl
      - run: python eval.py --model model.pkl \
        --min-f1 0.85 --max-regress 0.01 # gate
      - if: success() # promote
        run: python register.py --alias challenger
```

Ý chính: gate *fail* PR nếu data drift hoặc metric < threshold. Eval harness = Day 22; regression = §15; prod cần manual approval.

Lớp	Công cụ tiêu biểu (2026)	Hình thái
Orchestration	Kubeflow, Flyte, Metaflow, Airflow 3, Dagster, ZenML	OSS, tự host
End-to-end managed	SageMaker AI, Azure ML / Foundry, Databricks, Vertex→Gemini Ent. Agent Platform	managed (cloud)
Serving / inference	KServe (CNCf inc.), BentoML, Ray Serve, Dynamo	OSS + managed
Registry / tracking	MLflow 3.x, W&B, Comet	OSS + SaaS

Lưu ý: Đổi tên 2025–26: “Vertex AI” → **Gemini Enterprise Agent Platform** (04/2026); “SageMaker” = ô lớn, ML cổ điển = **SageMaker AI**; “Azure AI Studio” → **Microsoft Foundry**. Chọn theo lớp. **Serving license/ownership:** Seldon→BSL, Triton→Dynamo 1.0 GA, KServe CNCf-incubating — internals/telemetry §8 + Day 20.

MLOps — model bạn tự train

- **Artifact chính:** weights + dataset
- Train mỗi vòng; CI/CD/CT kích retrain
- **Metric:** accuracy / F1 / AUC (khách quan)
- **Test:** fixed test set, deterministic
- **Chi phí:** train-heavy, inference rẻ
- **Drift:** data/concept (§10)

LLMOps — foundation model có sẵn

- **Artifact chính:** prompt + context + RAG config (*thường không train*)
- Compose, không train; “prompt change = deploy”
- **Metric:** eval-as-primary, LLM-judge (§10)
- **Test:** golden set + output không xác định
- **Chi phí:** inference/token-heavy (§11, §8)
- **Drift:** +prompt / embedding / model-version (§15)

Compound AI System (Zaharia et al., Berkeley BAIR, 18/02/2024) — Hệ giải quyết task bằng *nhiều thành phần tương tác* — nhiều lần gọi model, retriever, tool, control-flow — *không* phải một model đơn.

- SOTA đến từ *kiến trúc hệ*, không chỉ scale model
- Kiểm soát: dễ vá 1 component hơn retrain
- Tin cậy + giới hạn chi phí ở tầng hệ

Lưu ý: Hệ quả vận hành: đối tượng deploy/version/eval không còn là “model” mà là cả *pipeline* (prompt+retriever+tool+model). Đó là lý do LLMOps ra đời.

“60% app RAG, 30% multi-step” là số minh họa một nguồn (Databricks, đầu 2024) — dùng định tính.

Lưu ý: **Bẫy:** temperature=0 *không* đảm bảo cùng output. **Batch size** đổi $\Rightarrow$ thứ tự cộng floating-point đổi $\Rightarrow$ kết quả khác (không phải lỗi seed).

Thinking Machines “batch-invariant kernels” (09/2025); SGLang deterministic mode (22/09/2025). Nhưng chi phí + không phải lúc nào cũng bật.

Quy tắc Ops — Đừng kỳ vọng output bit-identical. **Pin ngưỡng eval, không pin output:** gate trên “% pass $\geq$ X” với khoảng tin cậy (§10), không trên so-khớp-chuỗi.

Hệ quả: regression test LLM phải là *distributional*, chạy nhiều lần.

Prompt là artifact *deploy-critical* nhưng MLOps tools cũ không version nó. Prompt registry tách prompt khỏi code.

```
# Langfuse prompt registry (SDK v4, OTel-native)
prompt = langfuse.get_prompt("qa", label="production")
text = prompt.compile(question=q) # versioned + cached
# -> link prompt-version vào trace + eval score
```

Langfuse Prompt Mgmt, PromptLayer, Agenta, Latitude; LangSmith **Prompt Hub** (đổi tên từ “LangChain Hub”, versioning theo commit-hash).

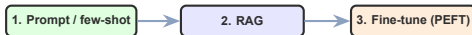
Lưu ý: Humanloop đã sunset (08/09/2025, Anthropic acqui-hire) — đừng phụ thuộc. Prompt phải có version, eval-score, rollback như code.

Embedding model có **version** → đổi = *re-embed corpus* (dual-index + cutover). Gemini Embedding 001 / Qwen3-Embedding-8B (đa ngữ); Matryoshka truncation. Vector DB ops = Day 19.

LightRAG/LazyGraphRAG cắt chi phí $\sim 100\text{--}6000\times$.
Agentic RAG: agent tự quyết retrieve/rerank (vòng lặp)
→ vận hành như agent (§19).

Cohere Rerank 3.5/v4, bge-reranker-v2-m3 (self-host). Two-stage: retrieve rộng → rerank top-k.
Cách vá RAG rẻ & hiệu quả nhất.

Reference-free eval (Ragas triad) — Faithfulness + Answer-relevancy + Context-precision/recall — không cần ground-truth, chạy online + CI + OTel (định nghĩa §10).



Khi prompt+RAG cạn: cần format/phong cách/latency, hoặc distill. *Không* để nhồi kiến thức (đó là RAG). **LoRA/PEFT** phủ ~95% nhu cầu (adapter nhỏ, rẻ); QLoRA = +quantization.

Multi-LoRA: 1 base + N adapter hot-swap (vLLM GA; LoRAX nghìn adapter) — tiết kiệm GPU (§8). **Post-training:** SFT → **DPO** (ship-default) → GRPO. Lý thuyết Day 22, serving Day 20.

Gateway = control plane —

1 proxy giữa app & N provider: routing+fallback, semantic cache, rate-limit + **budget/team**, virtual keys, audit, obs. Ref-arch: app→gateway→cache→router→model→eval→obs.

RouteLLM: câu dễ → model rẻ. Langfuse SDK v4
OTel-native → telemetry chung đường ray §7/§13.
FinOps accounting = §11.

Gateway

2026

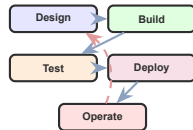
Portkey	OSS; Palo Alto mua (05/26); >1T tok/ngày
LiteLLM	Python, 100+ provider, MCP gateway
Kong AI GW	3.14; quản trị A2A
Cloudflare	edge GA + analytics

Latency: Python ms vs Go (Bifrost) ~11 μ s @ 5k rps.

AgentOps extends LLMOps —

Agent thêm: **autonomy + tool use + memory/loop.**

Lưu ý: Sai số nhân: $95\%/bước \times 20 \text{ bước} \approx 36\%$ cả task. Obs (§13) thấy lỗi; AgentOps ngăn (eval + reliability + governance + cost/loop).



Design tool/memory/guardrail · Test sim+eval-gate · Deploy durable+HITL · Operate trace(§13)+cost/loop. Vòng Operate→Design = nơi eval+governance đóng.

$\text{pass}@k$ vs pass^k — $\text{pass}@k$ = đúng ít nhất 1 trong k lần (lạc quan, cho research). pass^k = đúng cả k lần liên tiếp (đo độ tin cậy vận hành).

$\text{pass}@1$	k	pass^k (tin cậy)
90%	5	59%
90%	8	43%
99%	8	92%

Lưu ý: 90% $\text{pass}@1$ nghe “tốt” nhưng chỉ **43%** đáng tin ở k=8 lần lặp. Người dùng gặp pass^k , không phải $\text{pass}@1$.



Báo cáo pass^k (tau-bench) cho stakeholder; đặt SLO độ-tin-cậy, không chỉ accuracy đơn lần.

Benchmark	Đo gì	Cảnh báo 2026
τ 3-bench	tool-use hội thoại (banking/voice)	dùng pass^k, không pass@1
SWE-bench Verified	sửa bug repo thật	headline ~88–93% NHƯNG ICSE'26: ~19.78% “solved” bị reward-hack (cận trên)
GAIA	trợ lý đa bước	~74.6% (Sonnet 4.5) — phụ thuộc scaffold
BFCL v4	function-calling	top ~75%
WebArena	thao tác web	~70–74% (human 78.24%)

Lưu ý: Điểm benchmark *phụ thuộc scaffold* & có thể bị reward-hack → là *cận trên*, không phải độ tin cậy production. Luôn eval trên golden case của *bạn* (cross-ref §13).

Chạy agent với *user mô phỏng* + môi trường giả lập (tool mock) qua hàng trăm kịch bản trước khi prod. OpenAI “Deployment Simulation” (06/2026). Cần ≥ 500 **golden case** để có tín hiệu.

Re-chạy *trace production thật* (§13) qua agent mới $\rightarrow$ so trajectory/outcome (zero-cost, stale) trước khi rollout. Bổ sung cho shadow/canary (§15).



Agent test $\neq$ phần mềm test: output không xác định + có side-effect. Dùng simulation (kịch bản) + replay (trace thật) + eval-gate, không phải assert==.

- **Durable execution** (Temporal / Inngest / LangGraph): resume sau crash, replay-safe
- **Idempotency** cho tool side-effect (Day 12)
- Retry + circuit-breaker + timeout
- Checkpoint + HITL approval cho hành động rủi ro
- Eval-gate CI (Braintrust / LangSmith)
- Step-cap + budget (chống runaway)

Lưu ý: **Cost/loop control:** alert khi 1 run > step-cap; hash tool-call so rolling window (dup = no-progress, §13). Agent đốt $\sim 50\times$ token chat-bot; 1 runaway từng tốn \$12k (§13). FinOps accounting = §11.

Autonomy = error budget — Cho agent tự hành tới mức error-budget cho phép; vượt $\rightarrow$ hạ autonomy / chèn HITL.

Orchestration TRỞ THÀNH observability — Với run hàng giờ-ngày: **Temporal event-history** = distributed-trace + audit-log + *bảng replay*; deterministic replay re-chạy y hệt qua crash/deploy/HITL-pause. 2025–26: Workflow Streams (real-time obs) + Standalone Activities.

mỗi state-transition = checkpoint resume/fork-được; 1 run 10 bước ~100–200KB → vài xu/triệu run (storage gần như free).

Lưu ý: **Replay-against-new-model** = **regression gate** cho tác vụ nhiều giờ: re-chạy trace cũ qua model/harness *mới* → so trajectory/outcome trước rollout (zero-cost; nổi simulation/replay frame trên + shadow/canary §15).

Trace-identity sống qua process-death — stitch session/thread-id + state-diff theo node; observe *memory continuity* (giữ gì vs quên gì) như tín hiệu riêng, không chỉ per-call log.



“Bản upgrade model có phá vỡ workflow 4h mà bạn không dám chạy lại live?” — replay tape trả lời được, per-call log thì không.

- **Short-term:** context window (đắt, giới hạn)
- **Long-term:** vector/episodic store, gọi khi cần
- **Memory blocks:** khối có cấu trúc, chỉnh sửa được (Letta)

Tooling 2026 — **Letta** (đổi tên từ MemGPT), Mem0, Zep, LangMem.

Lưu ý: Memory drift (§13): agent dài hạn quên/mâu thuẫn context cũ. Ops: version memory schema, TTL/forget policy (PDPL §12 — “right to be forgotten”), giám sát $\text{cosine}(\text{state_now}, \text{state_t-N})$.

- **Async Tasks** (SEP-1686): tác vụ dài
- **CIMD** thay Dynamic Client Registration (SEP-991) + client-credentials + IdP SSO
- **Official MCP Registry** = **PREVIEW** (từ 09/2025), *chưa* GA — pin version

Lưu ý: **Tool bloat:** nạp tất cả schema làm độ chính xác chọn-tool sụp từ ~43% xuống <14%. Giải: **dynamic discovery** / Anthropic **Tool Search** (GA 02/2026, cắt ~85% token) — chỉ nạp tool liên quan.

Ops: MCP = cặp span client+server (§13). Tool registry + version + access-scope như API nội bộ.

Framework	2026	Định vị
Microsoft Agent Framework	1.0 GA (03/04/2026); gộp Semantic Kernel + AutoGen	.NET + Python; SK/AutoGen → maintenance
LangGraph	v1; durable, graph-based	control-flow rõ ràng + LangSmith
CrewAI	role-based multi-agent	nhanh để prototype
Google ADK	Agent Development Kit	gắn Gemini Enterprise
Claude Agent SDK	đổi tên từ “Claude Code SDK” (09/2025)	agent loop + tools

Lưu ý: Anthropic: “build simple, thêm phức tạp chỉ khi cần” — nhiều agent production chỉ là loop + tool, không cần framework nặng. AG2 = community fork của AutoGen. Chọn framework theo nhu cầu durable/multi-agent, không theo hype.

rủi ro mới: memory poisoning, tool misuse, excessive agency, privilege compromise, cascading multi-agent failures.

Audit log mọi action (who/what/when), per-agent NHI scoped-token, kill-switch, VN **PDPL 91/2025** cho data agent chạm (§12). Governance là phần “Ops” mà observability đơn thuần không phủ.

HITL approval gate — Hành động rủi ro (\$, xoá DB, gửi mail) cần *người duyệt* trước khi thực thi — chặn trước, không chỉ alert sau (Replit §13).



Vòng tròn khép kín: observe (§13) → eval + reliability + governance (AgentOps) → đó là cách đóng “khoảng trống eval” mà 48% đội còn thiếu.

AIOps — AI/ML ứng dụng VÀO vận hành IT & observability — thuật ngữ do **Gartner** đặt (~2016). Dùng ML, NLP, AI reasoning để *tự động hoá* monitoring, anomaly detection, event correlation, RCA, incident response.

- **Ingest** — metrics / logs / traces / events
- **Detect** — anomaly / forecast
- **Correlate & dedup** — gom & khử trùng lặp
- **RCA** — truy nguyên căn nguyên
- **Remediate** — khắc phục (auto / HITL)

- **AIOps** = AI để CHẠY ops
- **MLOps/LLMOps/AgentOps** = ops để CHẠY AI
- Gartner (2025) đổi tên thị trường → *Event Intelligence*

vs

Cùng chữ “Ops” nhưng ngược chiều: AIOps lấy AI *phục vụ* vận hành; MLOps/LLMOps/AgentOps lấy vận hành *phục vụ* AI.

- **Thống kê:** 3-sigma, EWMA, Holt-Winters; tách mùa vụ STL
- **ML:** Isolation Forest, autoencoder, Prophet
- Ngưỡng *tĩnh* gây nhiễu khi tải đổi

- **TimesFM 2.5** (Google), **Chronos-2** (Amazon), **Moirai** (Salesforce)
- *Zero-shot* forecast/anomaly — không train riêng từng metric
- Chronos-2: uni+multivariate+covariate

Dynamic / threshold-free alerting — Dự báo *dải kỳ vọng* (forecast band) theo mùa vụ; báo động khi lệch khỏi dải — thay ngưỡng tĩnh, tự thích ứng tải ngày/đêm/lễ.

0

TS foundation models phát hiện bất thường *zero-shot* trên metric mới, cắt phần lớn công tuning ngưỡng thủ công.

- Gom theo *topology* / *thời gian* / *ngữ cảnh*; dedup, suppression
- Hàng ngàn alert → vài incident *actionable*

- Tương quan đa nguồn (metric+log+trace+deploy) định vị căn nguyên
- LLM tóm tắt timeline + đề xuất giả thuyết

Lưu ý: Phần lớn MTTR mất ở khâu *hiểu chuyện gì đang xảy ra* (time-to-context), không phải khâu sửa — correlation + RCA tự động đánh đúng nút thắt này.

91

Vendor 2026: PagerDuty giảm noise tới **91%**, BigPanda **95%+**; nén hàng ngàn alert còn <100 actionable/ngày. NeuBird: 57% đội nói <30% alert là actionable.

AI *nhúng thẳng* vào nền tảng observability — từ trợ lý hỏi-đáp đến agent tự điều tra:

Datadog Bits AI

AI SRE agent: tự điều tra khi alert, đưa giả thuyết RCA *trước* khi on-call vào.

Dynatrace Davis CoPilot

Lớp GenAI trên causal+predictive AI; chat + đề xuất hành động.

Grafana Assistant + Sift

NL tạo dashboard/query; *Sift* tự chạy loạt kiểm tra khi incident.

New Relic AI

Truy vấn ngôn ngữ tự nhiên; giám sát agentic.

Honeycomb BubbleUp

Tìm outlier thống kê tương quan với lỗi; Query Assistant (NL).

Splunk ITSI / PagerDuty / incident.io

Correlation + AI SRE “always-on” điều tra incident.

2

Hai nhóm: (1) *copilot* đưa kết luận nhanh; (2) công cụ *phơi bày bằng chứng* để bạn tự kết luận — chọn theo mức tin cậy cần.

Graded autonomy — L0 manual → L1 correlation → L2 timeline → L3 single-shot RCA → L4 agentic multi-step → L5 closed-loop (remediate + người duyệt). Đa số đội 2026 còn ở **L0–L1**.

- Auto-investigate/RCA, tóm tắt timeline, định tuyến on-call
- Rút ngắn *time-to-context*, giảm toil — HITL

- Bounded actions + giới hạn *blast radius*, idempotency
- Policy gates + PR có người duyệt; pin model version; regression suite từ incident cũ

Lưu ý: Mẫu thất bại số 1: nhảy thẳng từ 0 lên *auto-remediation* trong 1 quý — thiếu telemetry, tri thức vận hành, governance. Chỉ lên L5 sau khi chạy sạch L4 vài tháng.

Lộ trình tăng dần (DO)

- ☒ Bắt đầu từ **1 bài toán** (alert-noise / incident mgmt) trên data sẵn có
- ☒ Thang: Reactive → Integrated → Analytical → Prescriptive → Automated
- ☒ VN/thực tế: quick-win giảm noise; OSS (Grafana Sift, HolmesGPT/K8sGPT) trên nền Day-13
- ☒ Đo *time-to-context*, không chỉ MTTR

Cạm bẫy (DON'T)

- ☐ Chặn đường số 1: data quality + integration kém — “rác vào, rác ra”
- ☐ Over-automation / blind trust: AI suy từ lịch sử, *mù* tình huống mới
- ☐ Hype vs thực tế: phần lớn DN vẫn ở giai đoạn đầu
- ☐ Phó mặc auto-remediation khi chưa có governance

1. **Stack 7 services:** Prometheus + Grafana + Loki + Jaeger + Alertmanager + OTEL Collector + FastAPI app — docker compose up trong ~60s
2. **Demo 1:** Locust load → Grafana 6-panel dashboard (RPS, P50/95/99, error rate, GPU util, tokens/sec, cost)
3. **Demo 2:** Kill app container → multi-window burn-rate alert fires → Alertmanager → Slack
4. **Demo 3:** Single trace flame graph: gateway → vector search → LLM inference → response. Tail-sampling kept this trace because P99 exceeded.
5. **Demo 4:** Ingest shifted dataset → Evidently PSI report → drift gauge updates → retrain trigger.

Mục tiêu: Build full 7-service Docker Compose observability stack: instrument FastAPI service với Prometheus + OTel + structlog, ship to Loki/Jaeger, dashboard 3 Grafana boards (overview, SLO burn-rate, cost), wire 2 multi-burn-rate alerts → Slack, integrate Day 19 Qdrant + Day 20 llama.cpp metrics.

Deliverable: 3 Grafana dashboards live + 2 alerts firing/resolving in Alertmanager + Jaeger trace screenshot + Evidently drift report + REFLECTION.md.

Thời gian: 2h

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 4 tín hiệu + LLM SLI; **GPU_UTIL** nói dối → SM_ACTIVE/MFU.
- 2 Prom 3.x histograms; columnar $\gg$ ELK; cardinality budget; profiling = tín hiệu 4.
- 3 SLO/burn-rate là math; eval-as-metric (HTTP-200 $\neq$ correct); agent outcome $\neq$ trace.
- 4 FinOps $\sim$ 17%; VN PDPL 91/2025; shadow $>$ A/B.
- 5 Trinity MLOps $\rightarrow$ LLMOps $\rightarrow$ AgentOps; obs là table-stakes (89%), **eval là khoảng trống (52%)** — chi tiết frame kế.

CI/CD/**CT** + registry *aliases* (@champion) + eval-gate; maturity Google-3/MS-5; hidden tech debt (CACE) vẫn đúng 2026; feature-store ops (Day 19).

pass^k không pass@k (90%→43%@8); reliability eng (durable exec + HITL + cost/loop); memory/MCP/A2A ops; governance (OWASP/NHI).

Compound system là đơn vị ops; **prompt-as-code** + eval-driven dev; RAGOps (rerank ROI cao); fine-tune ladder; gateway = control plane; temp=0 ≠ reproducible.

AI *vận hành* ops (vs ops-cho-AI): anomaly/forecast (TS foundation models), correlation + auto-RCA giảm noise, observability copilots, agentic SRE (HITL, autonomy L0–L5).

Lưu ý: Sợi chỉ xuyên suốt: observe → eval + reliability + governance + *AI-assisted ops*. Gartner: >40% dự án agentic huỷ trước 2027 vì *vận hành* kém, không vì model.

Ngày 24: Data Governance & Security

“RBAC, encryption, PII handling, compliance (GDPR/ISO 27001/PDPL 91/2025) — bảo vệ data nhạy cảm trong AI pipeline. Audit logs từ Day 23 sẽ feed vào Day 24 governance.”

- Hoàn thành
Day23-Track2-Observability-Lab/
(Docker Compose, 7 services)
- Cài đặt Presidio library (pre-lab N24)
- Đọc trước: Vietnam PDPL Law 91/2025 (xem GLOSSARY-day23 §12 cross-ref)

Hỏi & Đáp

Câu hỏi về quan sát (Prometheus 3.x/TSDB/GPU+vLLM/goodput/eBPF/profiling/OTel/drift/FinOps/agent-obs/harness) hoặc vận hành — MLOps (CI/CD/CT, registry, maturity) / LLMOps (prompt-as-code, RAGOps, fine-tune, gateway) / AgentOps (pass^k, MCP/A2A, frameworks, governance)?



Cảm ơn!

AICB-P2T2 · Ngày 23

Monitoring & Observability Stack

`lms.vinuni.edu.vn` · Slide & template trên LMS